NETWORKWALKS

PENETRATION TESTING REPORT

CYBERSECURITY B083D

Pentester Name	Sham Sunder
Program/Batch	Week 4: Pentesting Project
Cybersecurity Mentor	Waqas Karim, CCIE
Scope/Target	medirozahospital.com
Objective	Access patient portal and extract confidential patient lab reports
Document Classification	**CONFIDENTIAL**
Date of Assessment	October 3, 2026

1. Executive Summary

During the Week 4 Pentesting Project evaluation, I conducted a controlled vulnerability assessment of the target domain `medirozahospital.com`, focusing on the security of its patient application portal.

During the assessment, I identified a critical vulnerability in the authentication interface that allowed security controls to be bypassed, potentially exposing protected health information (PHI) to unauthorized access. I also examined the password protection used on extracted documents and found that the passwords were vulnerable to standard dictionary attacks.

This exercise helped me understand the importance of secure authentication, protecting sensitive patient data, and using strong passwords for encrypted files.

2. Scope and Methodology

2.1 Scope: The assessment was limited to the following target as agreed with the client.

Target domain: https://medirozahospital.com

2.2 Methodology: I followed a structured black-box penetration testing methodology consisting of four phases.

- **Reconnaissance:** Passive information gathering using publicly available information and web-based tools.
- **Vulnerability Identification:** Analysing the application behaviour to find weaknesses in authentication and input handling.
- **Exploitation:** Demonstrating the real impact of each vulnerability through controlled exploitation.
- **Documentation:** Current and Final phase, recording all findings, evidence, and remediation recommendations in this report.

2.3 Tools Used: As someone who is still learning, I used more tools than the ones listed below, but these were the most useful for this assessment.

#	Tool	Usage in current scenario
1	**nslookup**	Used to resolve the target domain and find its associated IP address.
2	**NMAP/ZENMAP**	Used to scan the identified IP and gather information about exposed ports and services, within the authorized scope
3	**waf00f**	Used to check whether the target website was protected by a Web Application Firewall (WAF).

4	**Browser Developer Tools/Tricks**	Used the Network tab to inspect HTTP requests and responses, and the Application tab to check browser storage for potentially exposed data.
5	**JTR Jumbo**	Used to extract password hashes from supported files and test password strength through dictionary-based password recovery in the authorized lab.
6	**qpdf**	Used to decrypt password-protected PDF files when the required password was available, allowing further document analysis.
7	**exiftool**	Used to extract metadata from PDF files, including document properties and other available metadata.

3. Finding and proof of exploitation

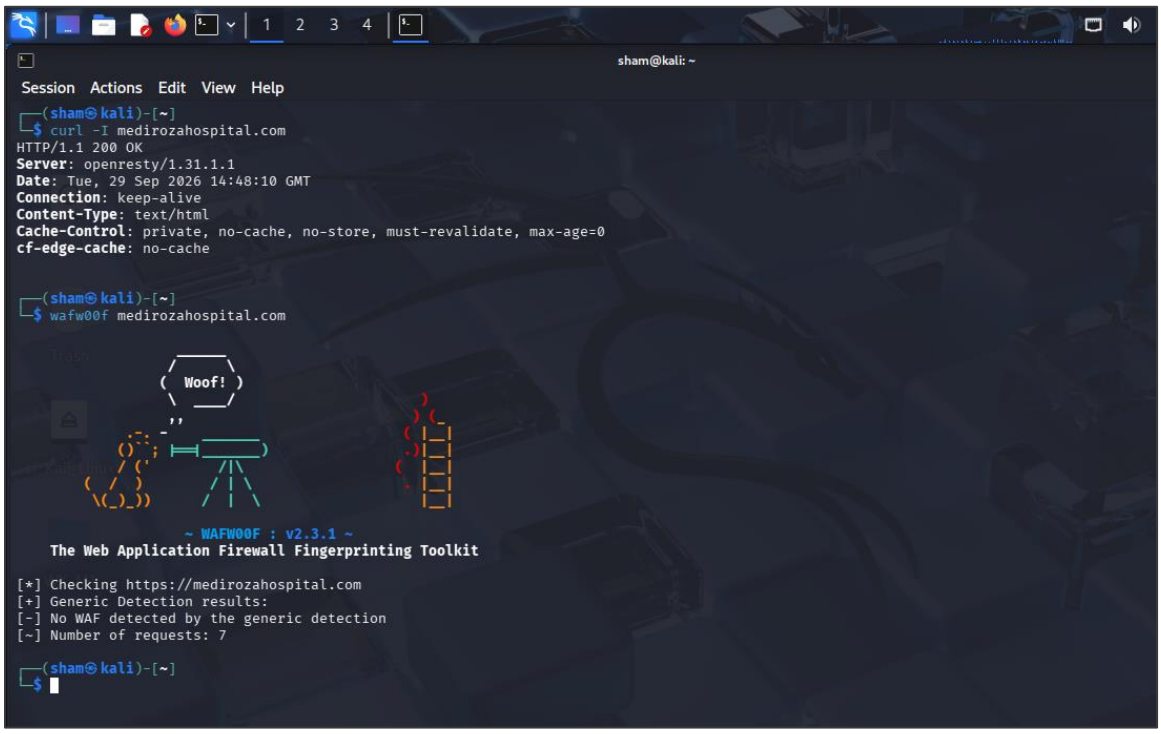
#	Vulnerabilty	Location	Risk
1	Missing Web Application Firewall (WAF)	/	Medium
2	Username Enumeration	/patient/login.php	Medium
3	Directory Listing	/patient/	Medium
4	SQL Injection Login Bypass	/patient/login.php	Critical
5	Exposure of Sensitive Files	/patient/portal.php/	High
6	Weak PDF Passwords	patient_report_*.pdf	High
7	Sensitive Metadata in Patient's PDF	patient_report_3.pdf	Medium
8	Forgotten backup folder with directory listing enabled	/old	Critical
9	Confidential Data in the /old Directory	/old/mediroza_db_backup_2019.sql	Critical

NOTE: "/" refers to the target website, home page.

3.1 Missing WAF | Location: “/” | Risk Level: Medium

Details: Not having a Web Application Firewall (WAF) is generally a security weakness, but it is not automatically a high-risk vulnerability on its own. The actual risk depends on the website's exposure, the security of its application, and the other controls in place.

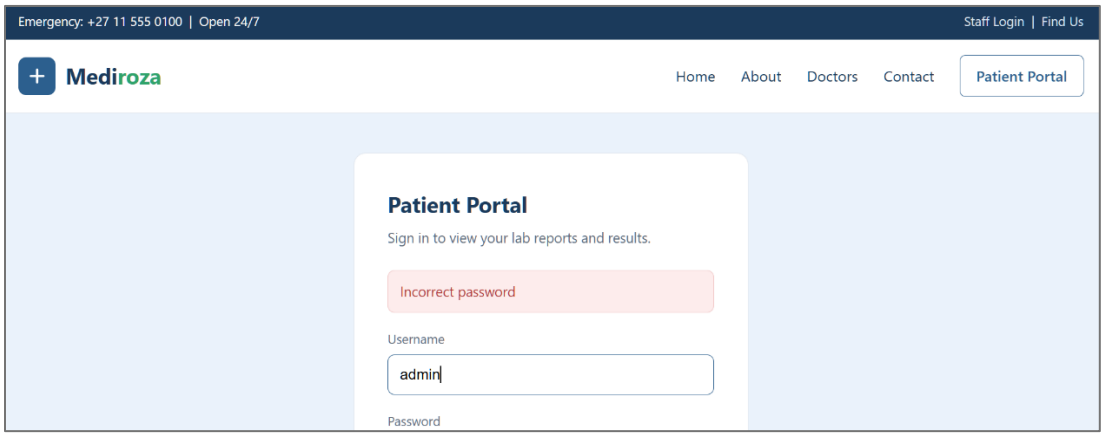
I opened my terminal in Kali Linux and ran: ‘waf00f medirozahospital.com’ and the output was: **No WAF detected by generic detection.**



3.2 Username Enumeration | Location: “/patient/login.php” | Risk Level: Medium

Details: Username enumeration happens when a login page reveals whether a username exists by showing different error messages for an incorrect username and an incorrect password. A secure login page should display the same generic error message in both cases so that an attacker cannot easily identify valid usernames.

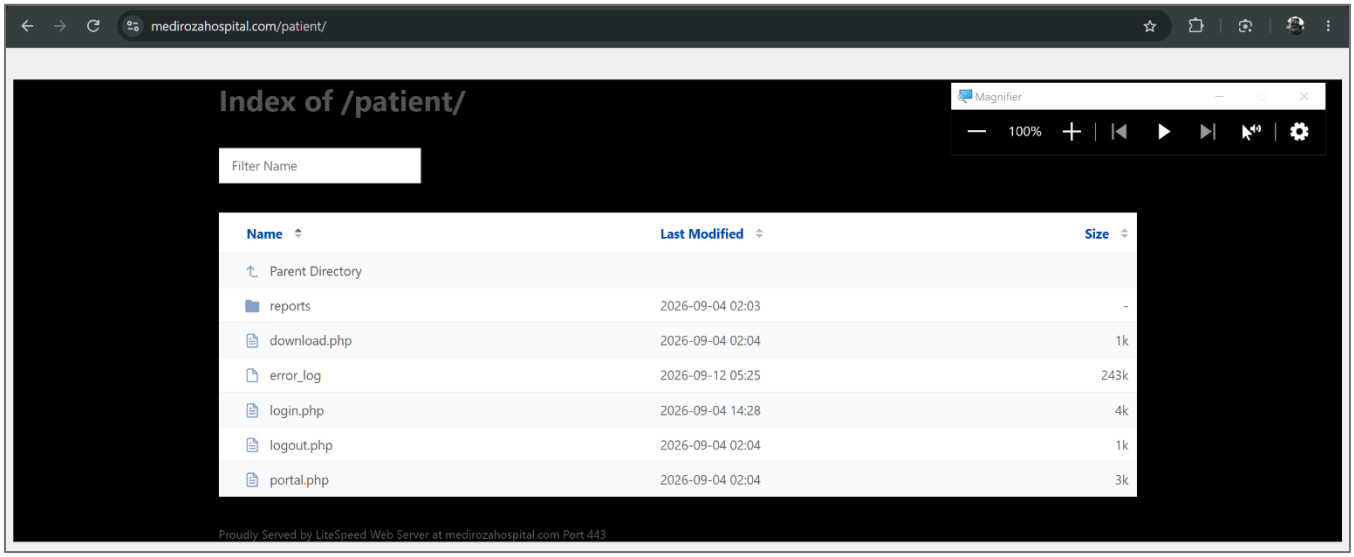
So, I opened the patients and staff login pages, and entered different usernames and passwords manually to observe the error messages, **staff** login portal was very ambiguous but **patient** portal error messages changed from **“incorrect username or password”** to **“incorrect password”** when I entered **admin** username:



3.3 Directory Listing | Location: “/patient/” | Risk Level: Medium

Details: Directory listing occurs when a web server allows users to view the files and folders inside a directory instead of restricting access to the intended application. In this case, directory listing was enabled on the patient page, which could expose files or other sensitive information to unauthorized users. Directory listing should be disabled, especially on directories containing patient or other confidential data.

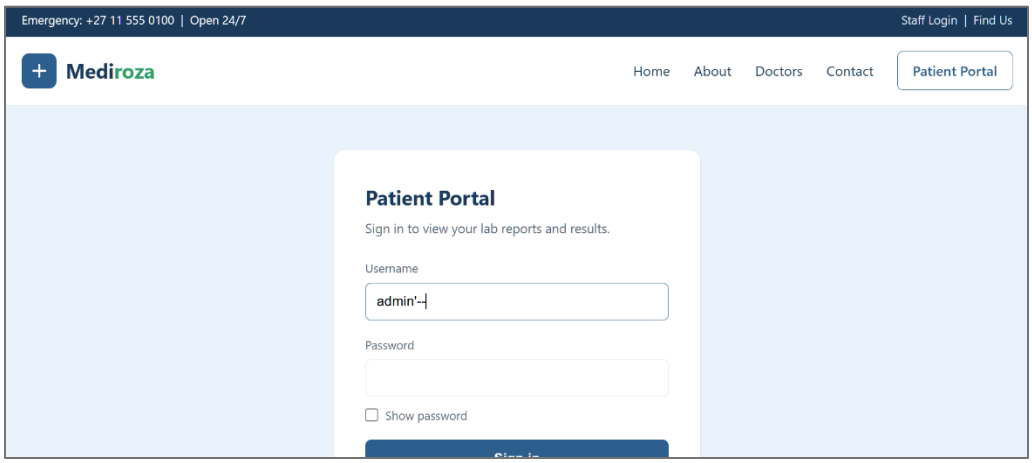
Risk level could become **High** if it exposes actual patient records or other confidential data, in this case it doesn’t because every time you try to open any page other than **login** it redirects you back to login.



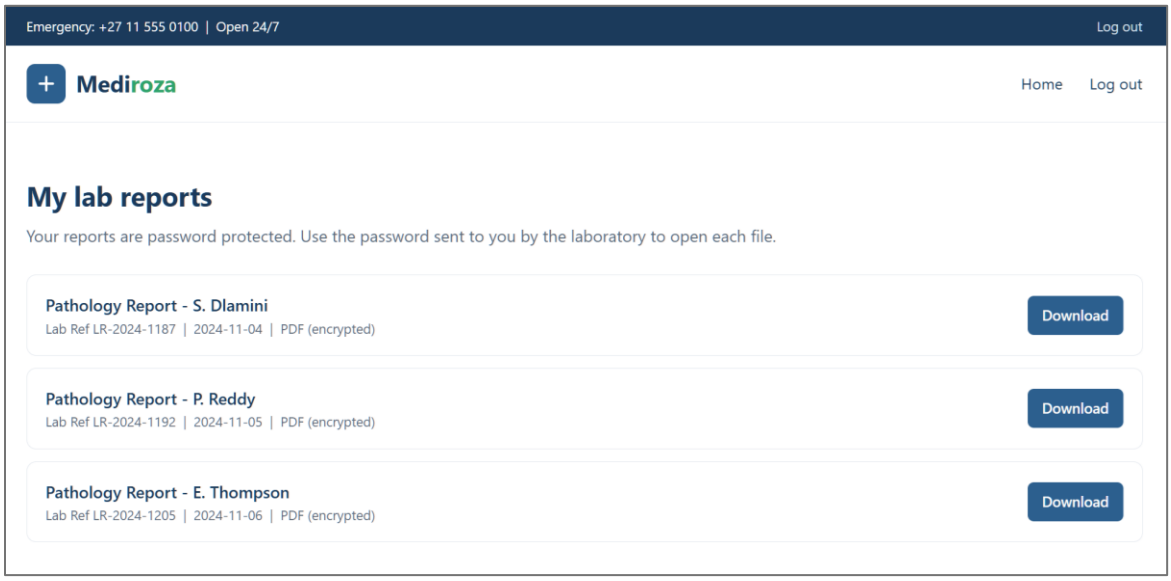
3.4 SQL Injection Login Bypass | Location: “/patient/login.php” | Risk Level: Critical

Details: SQL Injection is a vulnerability that occurs when an application does not properly validate or sanitize user input before using it in a database query. An attacker can manipulate the input to change how the query is processed. In this case, I was able to bypass the password check and log in as an administrator without knowing the correct password.

After getting the “incorrect password” on **admin** username I tried some common passwords, which didn’t work so I tried to look for some common **SQL Injections** and I came across this blog, which listed a bunch of common method, so I tried one **Line Comments,** by using the username **admin ‘--**:



Which means used **admin** as username and completely ignore the rest of the **SQL:**

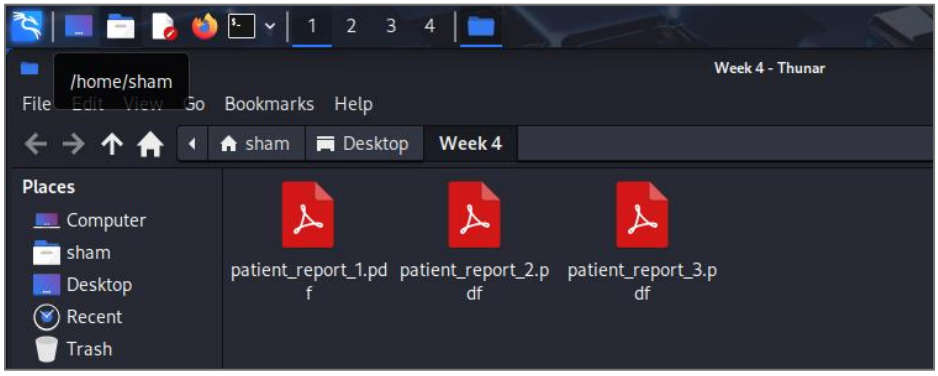


So I was logged into the portal and had **access to the patient reports.**

3.5 Exposure of Sensitive Files | Location: "/patient/portal.php" | Risk Level: High

Details: After bypassing the login through SQL injection, I found three patient lab report PDFs available for download. These documents contain confidential medical information and should only be accessible to the respective patients and their authorized doctors. Their availability to an unauthorized user indicates a serious issue with access control and the protection of sensitive patient data.

After logging in with the SQL injection payload I was directed to the patient portal, which listed three downloadable PDF files, which I was able to download:



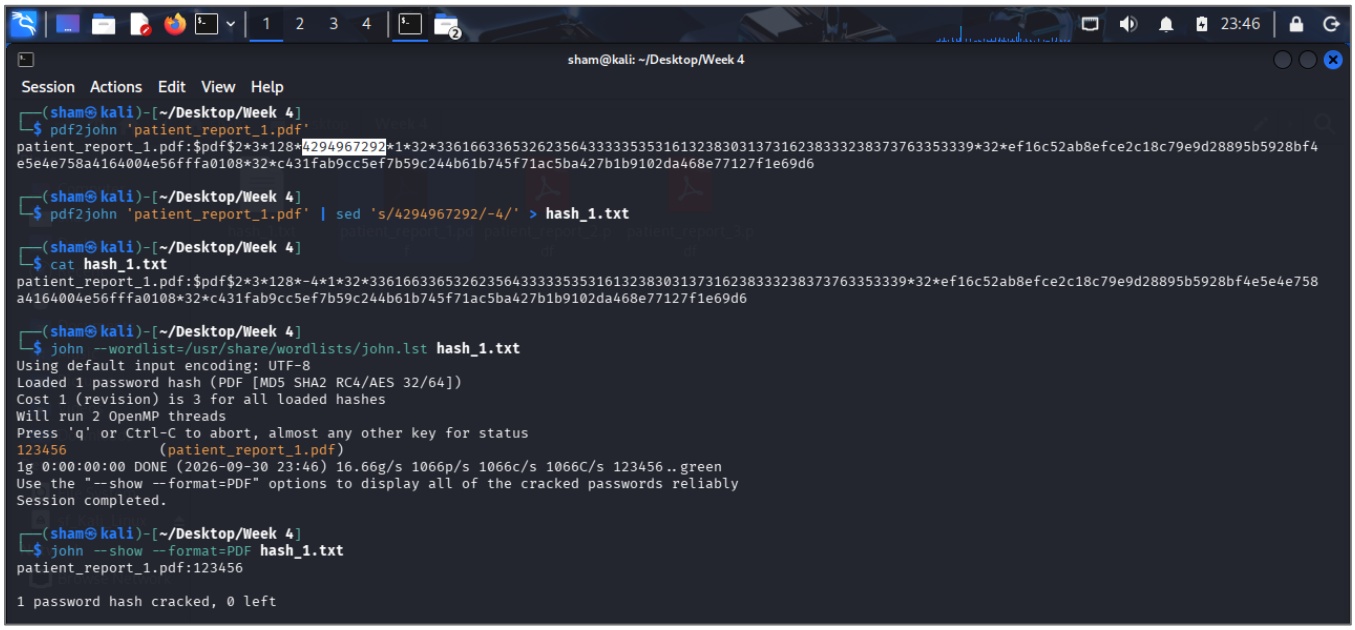
Which all required a password when I tried to open them, in short, they were **encrypted.**

3.6 Weak PDF Passwords | Location: "patient_report_*.pdf" | Risk Level: High

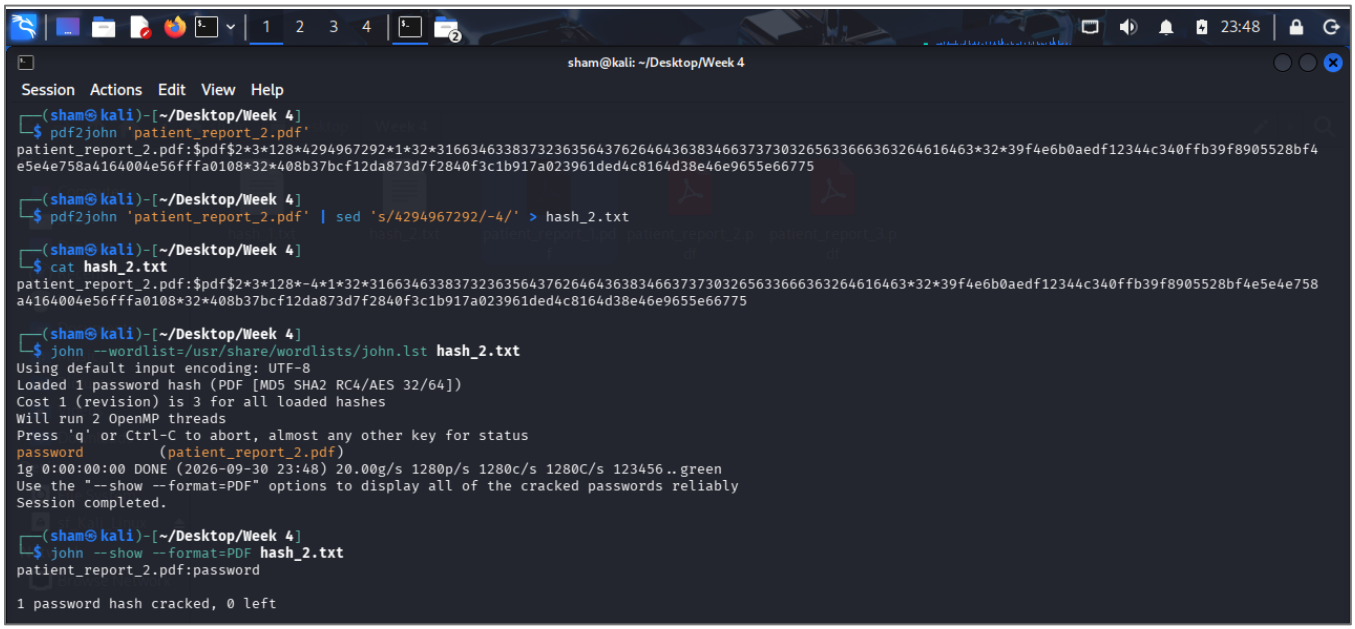
Details: The PDFs accessed through the patient portal were password-protected. However, the passwords were weak and appeared in standard password wordlists, making them easy to crack using automated tools. Using common passwords to protect sensitive medical documents provides little security against dictionary-based attacks.

I used **John the Ripper (JTR) Jumbo** for hash extraction, and John CLI to crack the passwords of PDF files, with the standard wordlist from the system **"/usr/share/wordlists/john.lst",** which was sufficient for all 3 files.

patient_report_1.pdf



patient_report_2.pdf



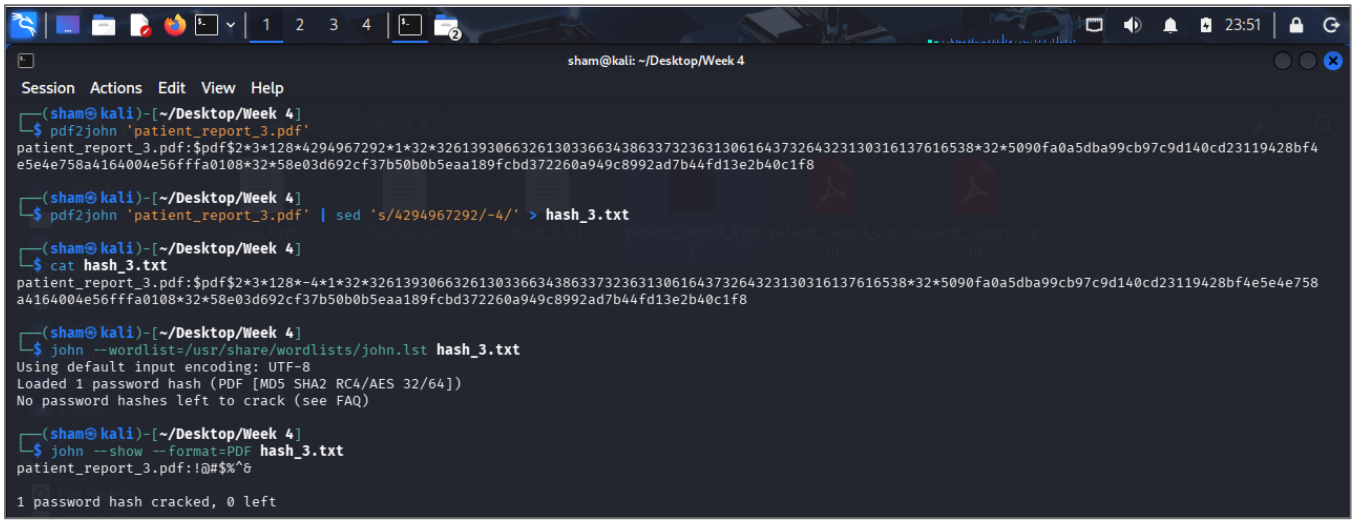
During password-cracking tests, John the Ripper (JtR) failed to recognize the hash generated by **pdf2john**. After investigating the issue, I found that the extracted hash contained the value **4294967292** in the PDF permissions field, whereas the expected signed representation was **-4**. This was a known issue (issue #5992) in **pdf2john**, which output the permissions value as an unsigned 32-bit integer instead of converting it to a signed integer.

Generated hash: pdf2*3*128*4294967292*1*32*...

Corrected hash: pdf2*3*128*-4*1*32*...

To resolve the issue, I used **sed** (Stream Editor) to replace **4294967292** with **-4** in the extracted hash before saving it to a text file. This corrected the permissions value's representation and allowed the hash to be recognized by the appropriate tool.

patient_report_3.pdf



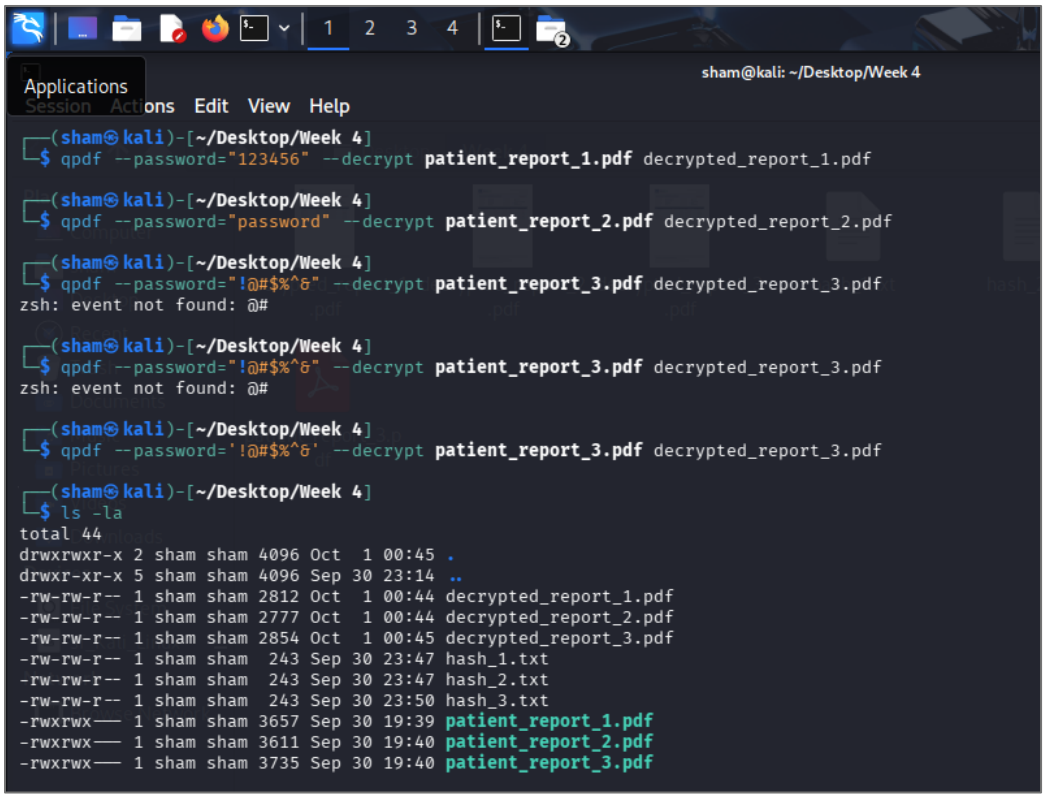
Report data:

Filename (.pdf)	Patient ID	Patient Name	Recovered Password
patient_report_1	MG-P-10231	Sipho Dlamini	123456
patient_report_2	MG-P-10244	Priya Reddy	password
patient_report_3	MG-P-10258	Emily Thompson	!@#$%^&

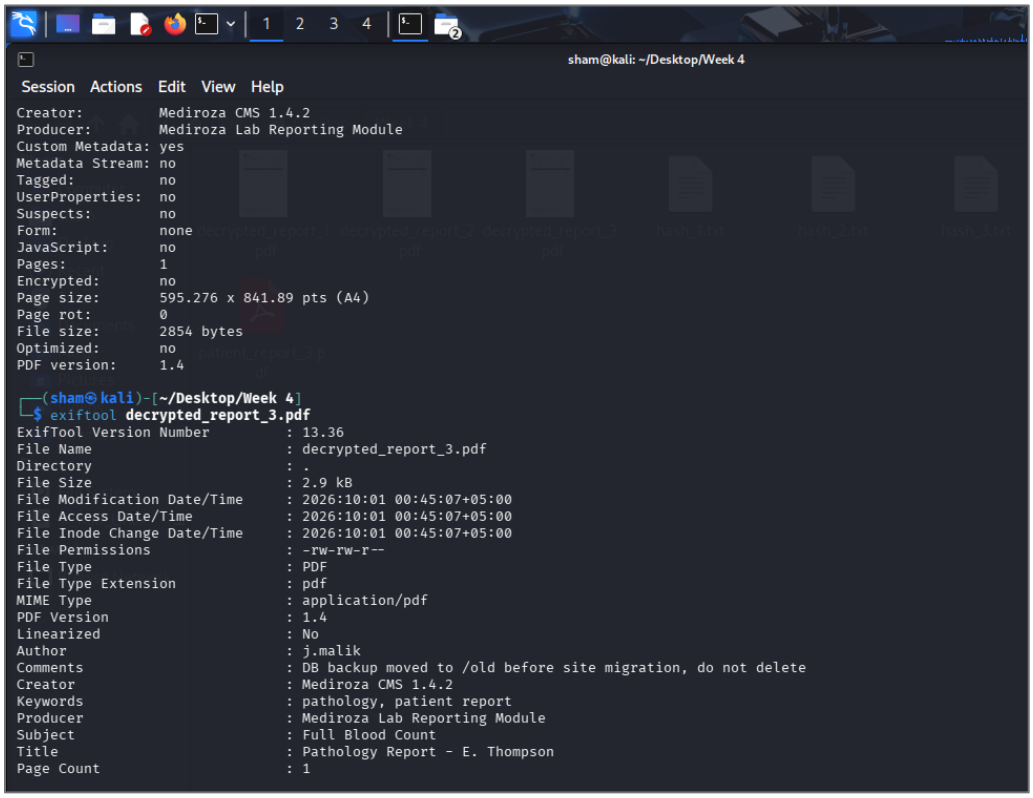
3.7 Sensitive Metadata in “patient_report_3.pdf” | **Risk Level:** Medium

Details: PDF files can contain hidden metadata such as the author name, creation date, and custom comments. This information is not normally visible when reading the document but can be extracted using tools such as exiftool. In this case, Report 3 contained an internal comment left by an IT administrator that referenced a confidential backup file stored on the server.

I first decrypted all the reports using **qpdf** so that **exiftool** could read all metadata fields. A locked PDF only shows the encryption type.



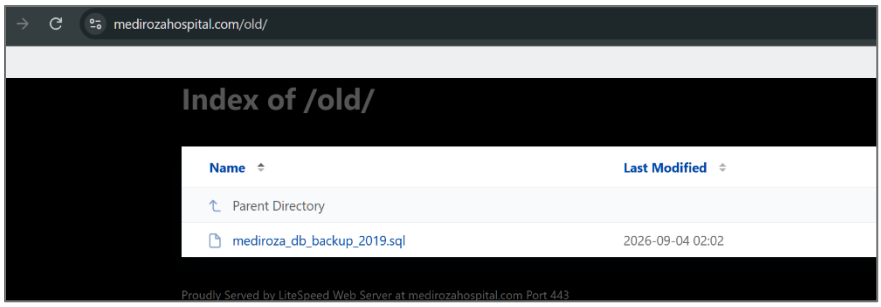
Then I ran exiftool on all 3 PDFs, 1 & 2 were clean, nothing unusual but report 3 showed author: **j.malik**, and a comment filed referencing a **/old** location:



NOTE: The comment was left by an IT Systems Administrator, Jameel Malik (**j.malik**). This was confirmed after inspecting the backup files located in the `/old` directory on the target website.

3.8 **Forgotten Backup Folder | Location:** /old **| Risk Level:** Critical

Details: Directory listing is a web server misconfiguration that displays the contents of a directory like a file browser when no index page is present. The **/old** directory on the server had directory listing enabled, exposing a database backup file (**mediroza_db_backup_2019.sql**) to anyone who accessed the URL.



I downloaded the backup file.

3.9 Confidential Data in the backup | /old/mediroza_db_backup_2019.sql | Critical

Details: The database backup contained highly sensitive information in plain text, including employees' full names, job titles, departments, email addresses, phone numbers, national ID numbers, and monthly salaries. It also contained the names of the hospital's shareholders and their respective shareholding percentages. This information was not protected by any access controls and was accessible to anyone who could access the **/old** directory.

I opened the SQL file in a **Chrome**. The data was in raw SQL format, containing **INSERT INTO** statements for the staff and shareholders tables.

Here is some of it:

```
-- mediroza_db_backup_2019.sql
-- Mediroza General Hospital - internal database backup
-- Generated by Mediroza CMS 1.4.2 backup module
-- Host: localhost    Database: mediroza_hr
-- WARNING: contains confidential staff and shareholder records
-- Backup date: 2019-08-27 02:14:03

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

--
-- Table structure for table `staff`
--
DROP TABLE IF EXISTS `staff`;
CREATE TABLE `staff` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `full_name` varchar(120) NOT NULL,
  `job_title` varchar(120) NOT NULL,
  `department` varchar(80) NOT NULL,
  `email` varchar(120) NOT NULL,
  `phone` varchar(20) NOT NULL,
  `national_id` varchar(20) NOT NULL,
  `monthly_salary_zar` int(11) NOT NULL,
  `date_joined` date NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
-- Dumping data for table `staff`
--
LOCK TABLES `staff` WRITE;
INSERT INTO `staff` (`id`,`full_name`,`job_title`,`department`,`email`,`phone`,`national_id`,`monthly_salary_zar`,`date_joined`) VALUES
(1,'Dr. Rajesh Naidoo','Chief Pathologist','Diagnostics Lab','r.naidoo@medirozahospital.com','+27 82 101 2007','85021013011081',138000,'2009-03-16'),
(2,'Sarah Botha','Chief Financial Officer','Finance','s.botha@medirozahospital.com','+27 82 102 2014','85031023022082',152000,'2011-07-01'),
(3,'Dr. Johan van der Merwe','Medical Director','Management','j.merwe@medirozahospital.com','+27 82 103 2021','85041033033083',160000,'2007-01-22'),
(4,'Dr. Anita Naicker','Consultant Cardiologist','Cardiology','a.naicker@medirozahospital.com','+27 82 104 2028','85051043044084',132000,'2012-09-10'),
(5,'Dr. Ahmed Kara','Consultant Physician','Internal Medicine','a.kara@medirozahospital.com','+27 82 105 2035','85061053055085',128000,'2013-02-18'),
(6,'Dr. Yusuf Cassim','Senior Registrar','Emergency & Trauma','y.cassim@medirozahospital.com','+27 82 106 2042','85071063066086',74000,'2018-05-04'),
(7,'Michael Roberts','HR Director','Human Resources','m.roberts@medirozahospital.com','+27 82 107 2049','85081073077087',96000,'2010-11-15'),
(8,'Susan Pretorius','HR Officer','Human Resources','s.pretorius@medirozahospital.com','+27 82 108 2056','85091083088088',32000,'2016-08-23'),
(9,'Jameel Malik','IT Systems Administrator','IT','j.malik@medirozahospital.com','+27 82 109 2063','85101093099080',58000,'2015-04-12'),
(10,'Thabo Molefe','Network Engineer','IT','t.molefe@medirozahospital.com','+27 82 110 2070','85111103110081',46000,'2017-10-02'),
(11,'Nomvula Khumalo','Registered Nurse','Emergency & Trauma','n.khumalo@medirozahospital.com','+27 82 111 2077','85121113121082',34000,'2016-01-19'),
(12,'Lerato Mokoena','Registered Nurse','Pediatrics','l.mokoena@medirozahospital.com','+27 82 112 2084','85011123132083',33000,'2017-06-07'),
```

4. Recommendation and Remediation

4.1 **Deploy a WAF (Web Application Firewall):** Configure the hospital's Web Application Firewall (WAF) with appropriate SQL injection detection and blocking rules to identify and block malicious requests containing common SQL injection patterns, such as **' OR 1=1** and **UNION SELECT** and **[username]'--**. The WAF should be regularly updated and monitored to ensure the rules remain effective against evolving attack techniques.

4.2 **Path the code for patient portal login:** Rewrite the portal's backend authentication code to use **parameterized queries**. This separates SQL commands from user-supplied input and prevents malicious input from being interpreted as SQL code. And enforce strict input validation on username and email fields by allowing only expected characters and formats. This provides an additional layer of protection against malicious input.

4.3 **Update PDF Encryption:** Use modern, strong PDF encryption standards such as **AES-256** instead of older encryption methods. Ensure that strong, unique passwords are used for sensitive documents and that encryption settings are configured to prevent unauthorized access.

4.4 **Remove exposed backup files from the web root:** Database backups should not be stored in publicly accessible directories such as /old. Move existing backups to a secure location outside the web root.

4.5 **Disable directory listing:** Disable directory indexing on the web server so that users cannot browse the contents of directories without an index page.

4.6 **Restrict access to backup files:** Implement authentication and access controls for any backup files that must remain accessible through the web server. Ideally, backups should not be directly accessible over HTTP/HTTPS at all.

4.7 **Protect sensitive database information:** Sensitive employee and shareholder information should be protected through appropriate access controls and encryption where applicable. Database backups should also be encrypted at rest.

4.8 **Remove unnecessary sensitive data from backups:** Review what information is included in backups and avoid storing unnecessary sensitive fields, particularly national ID numbers, phone numbers, and salary information, where they are not required.

4.9 **Implement secure backup storage and management:** Store backups in a dedicated backup system or protected storage location with appropriate permissions, access logging, and retention policies.

4.10 **Review exposed directories and files:** Conduct a web-server-wide review for other backup files, old application copies, configuration files, and forgotten directories that may be publicly accessible.